



IME III, PROGRAMACIÓN ORIENTADA A OBJETOS

Evaluación Primer Parcial

**Sistema de Gestión de Competencia de Robótica
Estudiantil**

Fecha de entrega y exposición: Viernes 11 de septiembre, 2026

1. Planteamiento del problema

La institución organizará una **Competencia de Robótica Estudiantil**, donde equipos inscriben robots para enfrentarse por categorías (tipo de robot). Actualmente no existe ningún sistema que permita gestionar esta información: los equipos, sus integrantes, los robots que presentan, las disciplinas en las que compiten y el resultado de los enfrentamientos se manejan "a mano", lo cual genera errores, pérdida de información y falta de trazabilidad.

Se les pide **diseñar y construir un Sistema de Gestión de Competencia de Robótica** que registre equipos, estudiantes y robots, inscriba a los robots en sus disciplinas, y — una vez cerrado el registro — genere automáticamente los enfrentamientos entre robots del mismo tipo, seleccionados y resueltos de forma aleatoria.

El objetivo educativo no es construir un sistema robusto ni con muchas funcionalidades. El objetivo es que sean capaces de:

- 1. Entender un problema real y **traducirlo a objetos**: qué entidades existen, qué datos y comportamientos tiene cada una, y cómo se relacionan entre sí.
- 2. Aplicar correctamente **encapsulamiento** y **mensajes entre objetos**, sin usar herencia (eso se verá en el Parcial 2).
- 3. Recorrer el **ciclo de vida de desarrollo de software** completo, no solo programar.

La calidad de la solución se mide principalmente por **qué tan bien está diseñado el sistema orientado a objetos**, no por cuántas funciones tiene.

2. Roles del equipo

Rol	Responsabilidad principal
Analista / líder de diseño	Entender el problema, identificar objetos/clases/relaciones, coordinar al equipo, validar que el diseño se respete en la implementación
Programador principal	Codificar las clases núcleo del sistema (según el diseño definido por el equipo) y el algoritmo de batallas
Programador(es) de módulos	Codificar inscripción, registro de disciplinas y flujo de interacción con el usuario (puede haber dos en equipos de seis)
Tester / QA	Diseñar y ejecutar casos de prueba, documentar bugs encontrados y su corrección
Documentador / presentador	Redactar el README, coordinar la documentación general y liderar la exposición final

Todos los integrantes deben codificar, sin importar su rol. El rol define quién lidera y es responsable de que la etapa se cumpla, no quién es "el único que programa".

3. Requerimientos funcionales

ID	Requerimiento
RF-01	Registrar N equipos: nombre del equipo, número de integrantes, y por cada integrante su nombre y carrera.
RF-02	Registrar, por cada equipo, sus robots: nombre y tipo (disciplina).
RF-03	El tipo de robot debe limitarse a un conjunto de valores válidos definido por el equipo (sugeridos: sumo, seguidor de línea, laberinto, velocista).
RF-04	Inscribir automáticamente cada robot a la disciplina correspondiente a su tipo.
RF-05	Cerrar el registro e iniciar la fase de competencia una vez registrados todos los equipos.
RF-06	Generar enfrentamientos por selección aleatoria de pares de robots del mismo tipo.
RF-07	Determinar el ganador de cada batalla de forma aleatoria (no por reglas de combate).
RF-08	Manejar el caso de un robot sin rival en su disciplina, reportándolo explícitamente (el equipo define y documenta si se descalifica, pasa directo, o queda "sin enfrentamiento").
RF-09	Generar un reporte final en consola: equipos, robots por equipo, y resultado de cada batalla.

Fuera de alcance: interfaz gráfica, base de datos, simulación física de combate.

4. Identificación de clases y relaciones

Esta identificación es parte de la evaluación. El planteamiento del problema (sección 1) y los requerimientos funcionales (sección 3) contienen toda la información necesaria para que el equipo determine, por sí mismo, qué clases conforman el sistema, qué atributos y comportamientos tiene cada una, y cómo se relacionan (pertenece a / contiene a / compite con).

Como guía de análisis, el equipo puede apoyarse en preguntas como:

- ¿Quién participa en la competencia y qué información se necesita de esa persona?
- ¿Qué agrupa a los participantes y sus robots?
- ¿Qué entidad describe a un robot y sus características?
- ¿Cómo se clasifica o categoriza a los robots para que compitan entre sí?
- ¿Qué entidad administra el enfrentamiento entre dos robots y su resultado?

Las clases identificadas deben colaborar entre sí mediante **mensajes** (llamadas a métodos), sin relaciones de herencia.

Sobre el diseño visual: todavía no hemos visto notación UML en clase, así que no se exige ningún formato específico. Sin embargo, **las clases y sus atributos deben quedar representados de forma visual** (rectángulos con nombre de clase y atributos, conectados con líneas que indiquen la relación: "pertenece a", "contiene a", "compite con"). Herramienta libre — papel, hoja de cálculo, imagen, editor de diagramas. Lo que se evalúa es que el equipo pueda **mostrar** su diseño antes de programarlo, no solo describirlo en texto.

5. Ejemplo de interacción con el usuario

```
=== Sistema de Gestión de Competencia de Robótica ===

¿Cuántos equipos se van a registrar? 2

--- Registro de Equipo 1 ---
Nombre del equipo: Los Engranés
¿Cuántos participantes tiene el equipo? 3

Integrante 1
  Nombre: Ana Torres
  Carrera: Ingeniería en Mecatrónica

Integrante 2
  Nombre: Luis Peña
  Carrera: Ingeniería Industrial

Integrante 3
  Nombre: Karla Ruiz
  Carrera: Ingeniería en Mecatrónica

¿Cuántos robots presenta el equipo "Los Engranés"? 2

Robot 1
  Nombre: Titán
  Tipo de robot (sumo / seguidor de línea / laberinto / velocista): sumo

Robot 2
  Nombre: Rayo
  Tipo de robot (sumo / seguidor de línea / laberinto / velocista): velocista

--- Registro de Equipo 2 ---
Nombre del equipo: Circuito Norte
...
```

(se repite el mismo flujo)

...

=== Registro completo ===

Equipos registrados: 2

Robots registrados: 4

=== Iniciando fase de competencia ===

Disciplina: Sumo

Batalla 1: Titán (Los Engranes) vs Bestia (Circuito Norte)

Ganador: Titán

Disciplina: Velocista

Robot Rayo (Los Engranes) no tiene rival registrado en su tipo.

=== Resultados finales ===

Sumo: Ganador -> Titán (Equipo: Los Engranes)

Velocista: Sin enfrentamiento (rival insuficiente)

Gracias por usar el Sistema de Gestión de Competencia de Robótica.

El caso de "Rayo" sin rival es intencional: obliga al equipo a decidir cómo manejarlo en su diseño (RF-08), y esa decisión debe quedar documentada.

6. Fases de desarrollo a cubrir

Fase	Qué debe evidenciarse	Responsable líder
Análisis	Entendimiento del problema y de los requerimientos (sección 3)	Analista / líder de diseño
Diseño	Diagrama visual de clases y relaciones (sección 4)	Analista / líder de diseño
Implementación	Código fuente en C++ siguiendo el diseño acordado	Programador principal + Programador(es) de módulos
Integración y pruebas	Casos de prueba ejecutados, bitácora de bugs encontrados/corregidos	Tester / QA
Documentación	README.md ejecutable + documento de diseño	Documentador / presentador
Mejora continua	Propuesta individual de nueva funcionalidad (no se implementa)	Cada estudiante, de forma individual

Estas fases se recorren de manera guiada en las sesiones de clase entre el 27 de agosto y el 10 de septiembre, con seguimiento del profesor, pero **no generan entregas calificables intermedias** — la entrega es única (sección 7).

7. Entrega única — 11 de septiembre, 12:30 PM

1. Repositorio de **GitHub** con el código fuente completo
2. **README.md** con instrucciones para compilar y ejecutar en **Windows y macOS**
3. Documento de diseño con representación visual de clases y atributos (sección 4)
4. Evidencia de pruebas (casos de prueba y bugs encontrados/corregidos)
5. **Propuesta individual de mejora continua** — una por cada integrante
6. Exposición del proyecto frente al grupo

8. Rúbrica de evaluación

Criterio	%	Qué se evalúa
Diseño orientado a objetos	50%	Clases correctamente identificadas (RF/sección 4), encapsulamiento, mensajes claros entre objetos, ausencia de herencia, coherencia entre el diagrama y el código
Cumplimiento funcional	20%	Cumplimiento de RF-01 a RF-09
Calidad de código y buenas prácticas	10%	Nombres claros, organización de archivos, comentarios donde aportan, manejo básico de errores
Trabajo en equipo y cumplimiento de rol	10%	Evidencia de commits distribuidos, cumplimiento de responsabilidades por rol (sección 2)
Documentación y exposición	10%	README ejecutable, exposición con participación individual, propuesta de mejora entregada por todos